

Lecture 7: Data Wrangling Continued

Heather Mattie, PhD

September 22, 2026

Recipe of the Day!

Potato Leek Soup



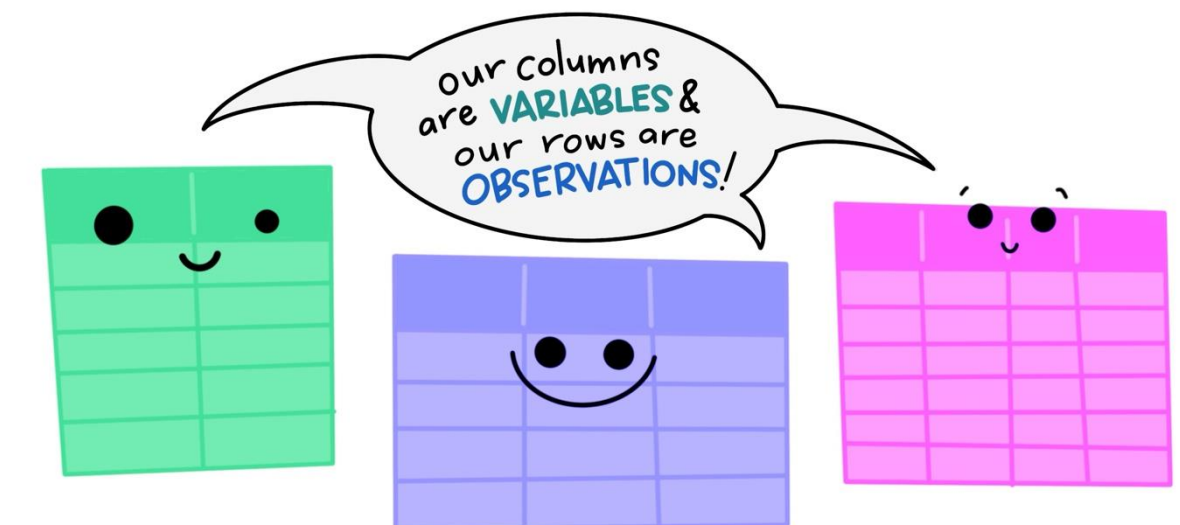
When you find the bug in your code and fix it on the first try

Announcements

- Heather's office hour this week: Thursday 12-1pm
- Homework #2 due Friday, October 9th by 11:59pm
- Lab will be held this week

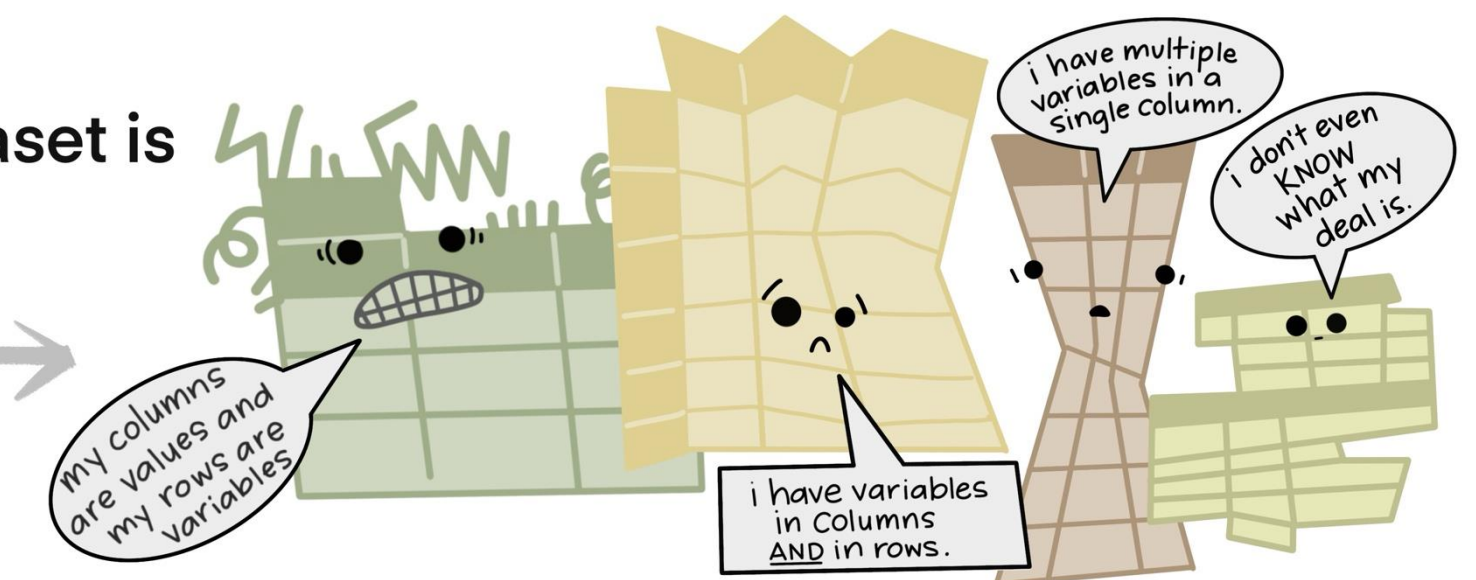


The standard structure of tidy data means that "tidy datasets are all alike..."



"...but every messy dataset is messy in its own way."

—HADLEY WICKHAM



Coding question of the day

Using the **gapminder** dataset, create a new data frame with only the **country**, **year**, **life_expectancy**, and **fertility** columns. Use the **pivot_longer()** function to reshape the dataset so that the **life_expectancy** and **fertility** columns are combined into a single value column called "**value**". The column names should be stored in a new column called "**health_indicator**".

Be sure to run this code first

```
# Load necessary libraries
library(dslabs)
library(tidyr)
library(dplyr)

# Load the gapminder dataset
data("gapminder")
```

Data Wrangling

Data Wrangling Roadmap

1. `dplyr` functions
2. Importing data
3. Reshaping data
4. Combining data frames
5. Parsing dates and times

Data Wrangling

Combining data frames

Heather Mattie, PhD

Multiple data sources

The data needed for analysis may not be in one table, file, or data frame

- Multiple data sources will need to be **combined** into one data frame
- Join functions, binding functions, set operators can be used

Name	City	Country
Lenna	San Francisco	US
Malcom	New York	US
Akiko	Tokyo	Japan

+

Name	City	Country
Lenna	San Francisco	US
Thomas	London	UK
Diane	Chicago	US



Name	City	Country
Malcom	New York	US
Akiko	Tokyo	Japan
Thomas	London	UK
Diane	Chicago	US

Example data frames

```
library(tidyverse)
library(dslabs)
data(murders)

tab1 <- slice(murders, 1:6) %>%
  select(state, population)
tab1
```

##	state	population
## 1	Alabama	4779736
## 2	Alaska	710231
## 3	Arizona	6392017
## 4	Arkansas	2915918
## 5	California	37253956
## 6	Colorado	5029196

```
data(polls_us_election_2016)

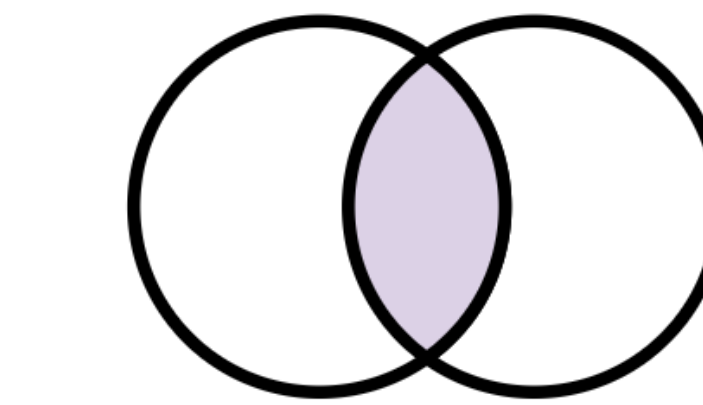
tab2 <- slice(results_us_election_2016, c(1:3, 5, 14, 44)) %>%
  select(state, electoral_votes)
tab2
```

##	state	electoral_votes
## 1	California	55
## 2	Texas	38
## 3	Florida	29
## 4	Illinois	20
## 5	Arizona	11
## 6	Alaska	3

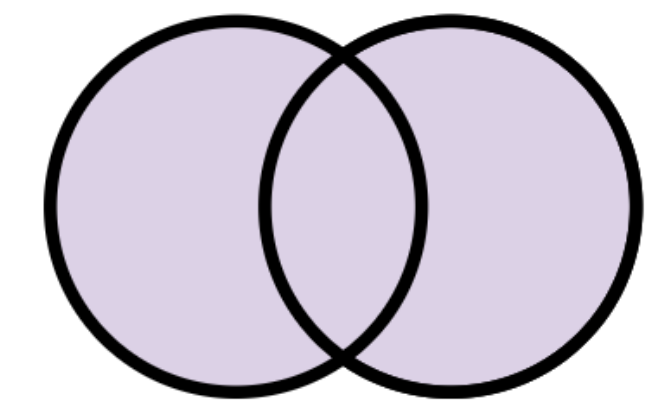
Join functions

Join functions can combine two data frames in different ways

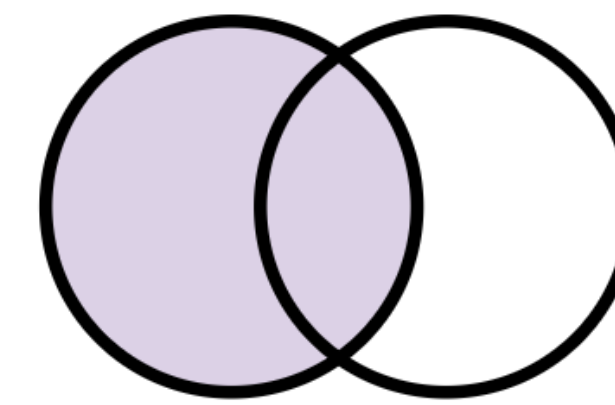
- The **first** data frame mentioned in the code is the **left** data frame
- The **second** data frame mentioned in the code is the **right** data frame



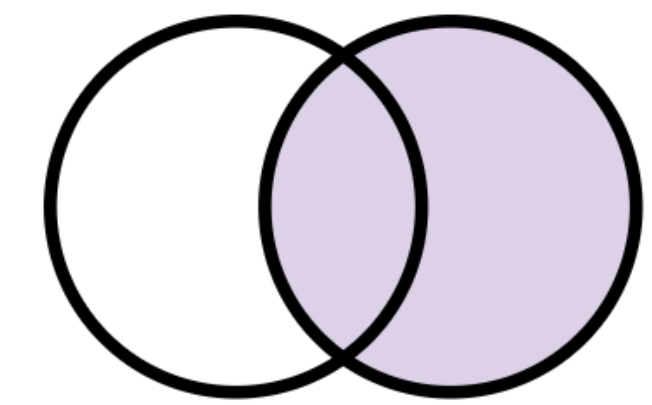
`inner_join(x, y)`



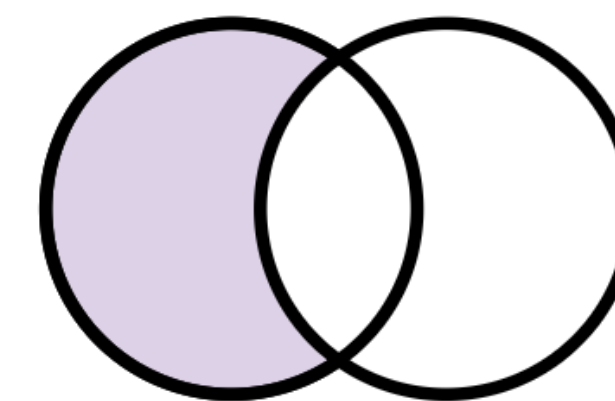
`full_join(x, y)`



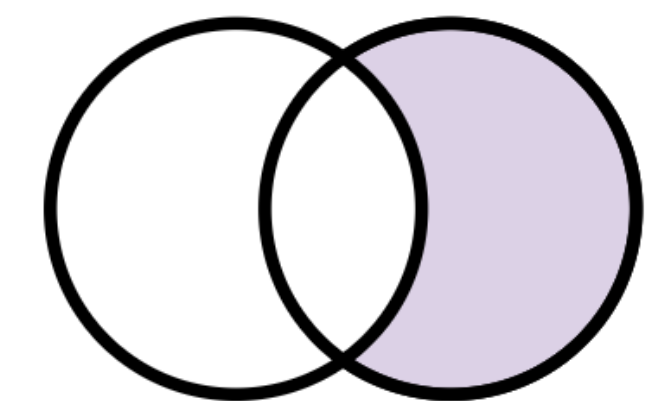
`left_join(x, y)`



`right_join(x, y)`

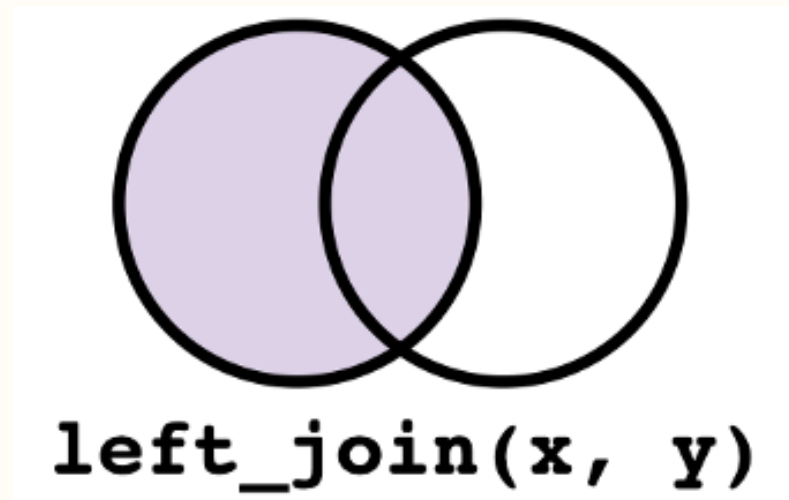


`anti_join(x, y)`



`anti_join(y, x)`

left_join



The **left_join** function retains all rows from the “left” data frame and matches rows from the “right” data frame

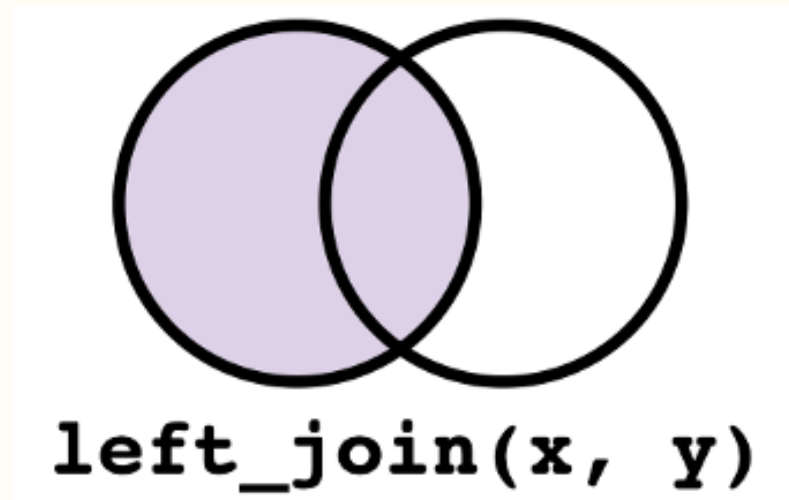
- Based on a specified common column
- If there is no match, an NA value will be produced

```
left_join(tab1, tab2)
```

```
## Joining with `by = join_by(state)`
```

##	state	population	electoral_votes
## 1	Alabama	4779736	NA
## 2	Alaska	710231	3
## 3	Arizona	6392017	11
## 4	Arkansas	2915918	NA
## 5	California	37253956	55
## 6	Colorado	5029196	NA

left_join



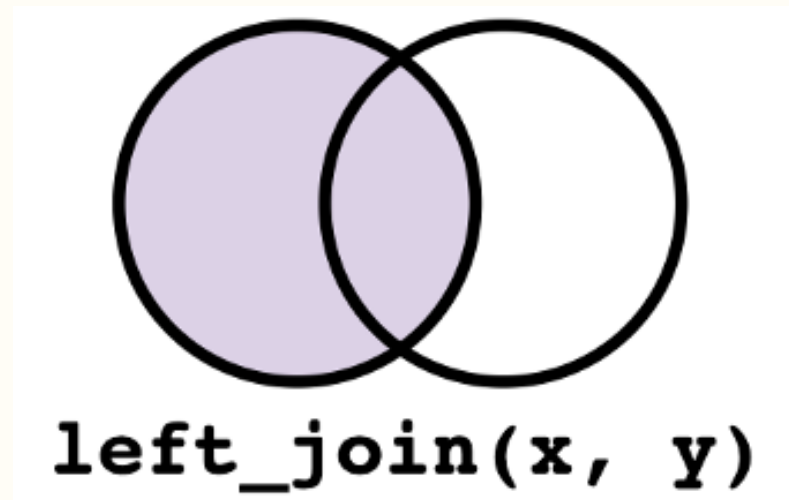
The **left_join** function retains all rows from the “left” data frame and matches rows from the “right” data frame

- Based on a specified common column
- If there is no match, an NA value will be produced

```
left_join(tab1, tab2, by = "state")
```

##	state	population	electoral_votes
## 1	Alabama	4779736	NA
## 2	Alaska	710231	3
## 3	Arizona	6392017	11
## 4	Arkansas	2915918	NA
## 5	California	37253956	55
## 6	Colorado	5029196	NA

left_join



The **left_join** function retains all rows from the “left” data frame and matches rows from the “right” data frame

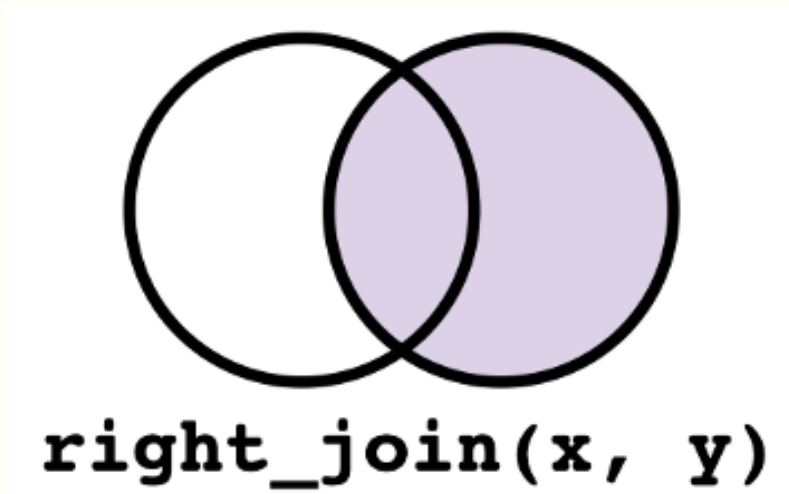
- Based on a specified common column
- If there is no match, an NA value will be produced

```
tab1 %>% left_join(tab2)
```

```
## Joining with `by = join_by(state)`
```

##		state	population	electoral_votes
## 1		Alabama	4779736	NA
## 2		Alaska	710231	3
## 3		Arizona	6392017	11
## 4		Arkansas	2915918	NA
## 5		California	37253956	55
## 6		Colorado	5029196	NA

right_join



The **right_join** function retains all rows from the “right” data frame and matches rows from the “left” data frame

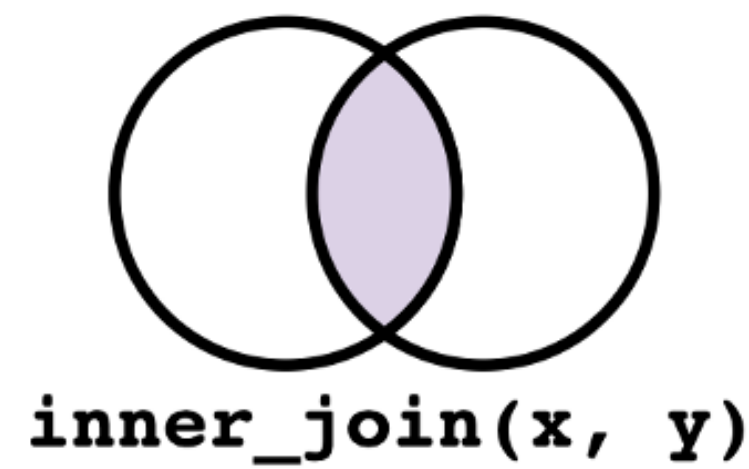
- Based on a specified common column
- If there is no match, an NA value will be produced

```
tab1 %>% right_join(tab2)
```

```
## Joining with `by = join_by(state)`
```

```
##      state population electoral_votes
## 1  Alaska      710231             3
## 2  Arizona     6392017            11
## 3 California  37253956            55
## 4   Texas           NA             38
## 5  Florida           NA             29
## 6 Illinois           NA             20
```


inner_join



The **inner_join** function retains all rows that have information in both data frames

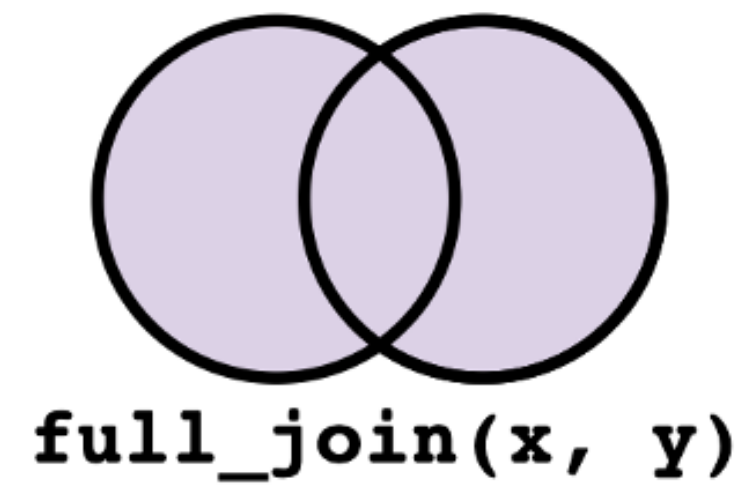
- Based on a specified common column
- Can be thought of as the intersection of the two data frames

```
inner_join(tab1, tab2)
```

```
## Joining with `by = join_by(state)`
```

```
##      state population electoral_votes
## 1   Alaska    710231             3
## 2  Arizona    6392017            11
## 3 California 37253956            55
```

full_join



The **full_join** function retains all rows from both data frames

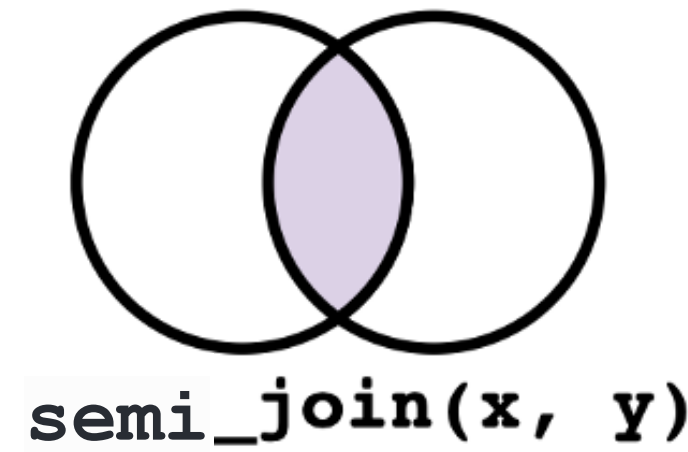
- Based on a specified common column
- If there is no match, an NA value will be produced

```
full_join(tab1, tab2)
```

```
## Joining with `by = join_by(state)`
```

##	state	population	electoral_votes
## 1	Alabama	4779736	NA
## 2	Alaska	710231	3
## 3	Arizona	6392017	11
## 4	Arkansas	2915918	NA
## 5	California	37253956	55
## 6	Colorado	5029196	NA
## 7	Texas	NA	38
## 8	Florida	NA	29
## 9	Illinois	NA	20

semi_join



The **semi_join** function retains all rows from the “left” data frame for which we have information in the “right” data frame

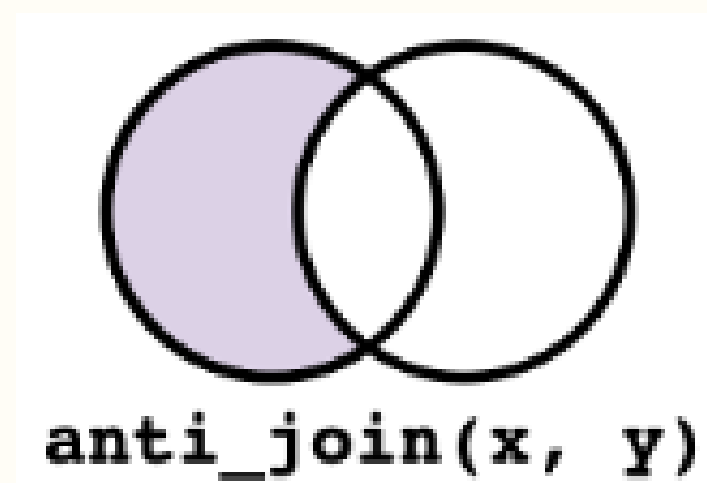
- Based on a specified common column
- No columns from the “right” data frame are retained

```
semi_join(tab1, tab2)
```

```
## Joining with `by = join_by(state)`
```

```
##      state population
## 1   Alaska    710231
## 2  Arizona    6392017
## 3 California 37253956
```


anti_join



The **anti_join** function retains all rows from the “left” data frame for which we have no information in the “right” data frame

- Based on a specified common column
- No columns from the “right” data frame are retained

```
anti_join(tab1, tab2)
```

```
## Joining with `by = join_by(state)`
```

```
##      state population
## 1  Alabama    4779736
## 2  Arkansas    2915918
## 3  Colorado    5029196
```

Binding

Binding is another way to combine data frames

- Binding functions do not match data frames by a common variable
- If the datasets don't match by the appropriate dimensions, an error occurs

x			y								
A	B	C				E	F	G			
a	t	1				a	t	3			
b	u	2				b	u	2			
c	v	3				d	w	1			

+

A	B	C	E	F	G
a	t	1	a	t	3
b	u	2	b	u	2
c	v	3	d	w	1

=

A	B	C	E	F	G
a	t	1	a	t	3
b	u	2	b	u	2
c	v	3	d	w	1

x			y								
A	B	C				A	B	C			
a	t	1				c	v	3			
b	u	2				d	w	4			

+

DF	A	B	C
x	a	t	1
x	b	u	2
y	c	v	3
y	d	w	4

bind_cols

The **bind_cols** function returns data frames placed side by side as a single table

- More than two data frames can be combined
- All data frames must have the same number of rows
- Does not match by column values

```
library(tidyverse)
library(dslabs)
data(murders)
head(murders)
```

```
##      state abb region population total
## 1  Alabama  AL  South    4779736    135
## 2  Alaska   AK   West     710231     19
## 3  Arizona  AZ   West    6392017    232
## 4  Arkansas AR  South    2915918     93
## 5 California CA   West   37253956   1257
## 6  Colorado CO   West    5029196     65
```

```
tab1 <- murders[, 1:2]
tab2 <- murders[, 3:4]
new_tab <- bind_cols(tab1, tab2)
head(new_tab)
```

```
##      state abb region population
## 1  Alabama  AL  South    4779736
## 2  Alaska   AK   West     710231
## 3  Arizona  AZ   West    6392017
## 4  Arkansas AR  South    2915918
## 5 California CA   West   37253956
## 6  Colorado CO   West    5029196
```


bind_rows

The **bind_rows** function returns data frames placed one on top of another as a single table

- More than two data frames can be combined
- All data frames must have the same number and names of columns
- Does not match by column values

```
tab1 <- murders[1:2,]  
tab2 <- murders[3:4,]  
bind_rows(tab1, tab2)
```

##		state	abb	region	population	total
## 1		Alabama	AL	South	4779736	135
## 2		Alaska	AK	West	710231	19
## 3		Arizona	AZ	West	6392017	232
## 4		Arkansas	AR	South	2915918	93

Set operators

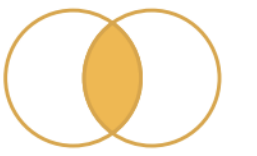
Set operators are another set of commands that are useful for combining data frames

- Here we will see the **intersect**, **union**, **setdiff**, and **setequal** functions in action
- When the **tidyverse** or **dplyr** packages are loaded these operators can be used on data frames rather than just vectors

A	B	C
c	v	3

intersect(x, y, ...)

Rows that appear in both x and y.



A	B	C
a	t	1
b	u	2

setdiff(x, y, ...)

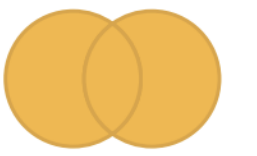
Rows that appear in x but not y.



A	B	C
a	t	1
b	u	2
c	v	3
d	w	4

union(x, y, ...)

Rows that appear in x or y, duplicates removed). **union_all()** retains duplicates.



Use **setequal()** to test whether two data sets contain the exact same rows (in any order).

intersect

The **intersect** function returns the elements that are common to two vectors, or the rows that are the same in two data frames with the same column names

```
intersect(1:10, 6:15)
```

```
## [1] 6 7 8 9 10
```

```
intersect(c("a","b","c"), c("b","c","d"))
```

```
## [1] "b" "c"
```

```
tab1 <- murders[1:5,]  
tab2 <- murders[3:7,]  
intersect(tab1, tab2)
```

```
##      state abb region population total  
## 1  Arizona  AZ   West    6392017    232  
## 2  Arkansas AR   South   2915918     93  
## 3 California CA   West   37253956   1257
```


union

The **union** function returns all elements that are in two vectors, or all rows that are in two data frames with the same column names

```
union(1:10, 6:15)

## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

union(c("a","b","c"), c("b","c","d"))

## [1] "a" "b" "c" "d"

tab1 <- murders[1:5,]
tab2 <- murders[3:7,]
union(tab1, tab2)

##      state abb  region population total
## 1  Alabama  AL   South    4779736    135
## 2  Alaska   AK   West     710231     19
## 3  Arizona  AZ   West     6392017    232
## 4  Arkansas AR   South    2915918     93
## 5  California CA   West    37253956  1257
## 6  Colorado CO   West     5029196     65
## 7  Connecticut CT Northeast  3574097     97
```


setdiff

The **setdiff** function returns elements that appear in a first vector but not in a second, or all rows that are in a first data frame but not in a second data frame with the same column names

- **setdiff** is not symmetric

```
setdiff(1:10, 6:15)
```

```
## [1] 1 2 3 4 5
```

```
setdiff(6:15, 1:10)
```

```
## [1] 11 12 13 14 15
```

```
tab1 <- murders[1:5,]  
tab2 <- murders[3:7,]  
setdiff(tab1, tab2)
```

```
##      state abb region population total  
## 1 Alabama  AL  South    4779736    135  
## 2  Alaska  AK   West     710231     19
```

setequal

The **setequal** function returns TRUE if two vectors are the same, regardless of order, or if two data frames are equal, regardless of order

```
setequal(1:5, 1:6)
```

```
## [1] FALSE
```

```
setequal(1:5, 5:1)
```

```
## [1] TRUE
```

```
setequal(tab1, tab2)
```

```
## [1] FALSE
```

Summary

- It is common to need to combine multiple datasets into one
- Join functions match data frames by a common variable
- Binding functions do not match data frames by a common variable, but do need appropriate dimensions to match
- Set operators can be applied to both vectors and data frames

Data Wrangling

Dates and times

Heather Mattie, PhD

Parsing dates and times

Dates are often encountered in data science projects and are their own **data type**

- A date can be stored as a string but can be converted to a number once a reference day (epoch) is chosen
- Computer languages typically use January 1st, 1970 as the epoch

```
library(tidyverse)
library(dslabs)
data("polls_us_election_2016")
head(polls_us_election_2016$startdate)
```

```
## [1] "2016-11-03" "2016-11-01" "2016-11-02" "2016-11-04" "2016-11-03"
## [6] "2016-11-03"
```

```
class(polls_us_election_2016$startdate)
```

```
## [1] "Date"
```

```
as.numeric(polls_us_election_2016$startdate) %>% head
```

```
## [1] 17108 17106 17107 17109 17108 17108
```

The **lubridate** package

The **lubridate** package contains functions that are useful and intuitive for working with dates and times

- There are functions for parsing, extracting, manipulating, and performing calculations on date and time data



month, day, and year functions

The **month**, **day**, and **year** functions can be used to extract the values of the month, day, and year from a date

```
library(lubridate)
set.seed(2)
dates <- sample(polls_us_election_2016$startdate, 10) %>% sort
dates
```

```
## [1] "2016-01-19" "2016-08-06" "2016-08-26" "2016-09-09" "2016-09-14"
## [6] "2016-09-16" "2016-09-29" "2016-10-04" "2016-10-12" "2016-10-23"
```

```
data.frame(month = month(dates),
            day = day(dates),
            year = year(dates))
```

```
##      month day year
## 1         1  19 2016
## 2         8   6 2016
## 3         8  26 2016
## 4         9   9 2016
## 5         9  14 2016
## 6         9  16 2016
## 7         9  29 2016
## 8        10   4 2016
## 9        10  12 2016
## 10       10  23 2016
```


month, day, and year functions

The **month**, **day**, and **year** functions can be used to extract the values of the month, day, and year from a date

- The month labels can also be extracted with the argument “**label = TRUE**” in the **month** function

```
month(dates, label = TRUE)
```

```
## [1] Jan Aug Aug Sep Sep Sep Sep Oct Oct Oct  
## 12 Levels: Jan < Feb < Mar < Apr < May < Jun < Jul < Aug < Sep < ...  
< Dec
```


Date parser functions

Another set of useful functions are the **parsers** that convert strings into dates

```
x <- c(20090101, "2009-01-02", "2009 01 03", "2009-1-4",  
       "2009-1, 5", "Created on 2009 1 6", "200901 !!! 07")  
ymd(x)
```

```
## [1] "2009-01-01" "2009-01-02" "2009-01-03" "2009-01-04" "2009-01-05"  
## [6] "2009-01-06" "2009-01-07"
```

Date parser functions

Any order of month, day, and year is possible with the date parser functions

- The preferred order is **YYYY-MM-DD** so if the string is ordered it will be ordered by date
- If the original order is unknown, it is useful to look at all values in a vector to determine which format the strings are in

```
x <- "09/01/02"  
ymd(x)
```

```
## [1] "2009-01-02"
```

```
ydm(x)
```

```
## [1] "2009-02-01"
```

```
myd(x)
```

```
## [1] "2001-09-02"
```

```
mdy(x)
```

```
## [1] "2002-09-01"
```

```
dmy(x)
```

```
## [1] "2002-01-09"
```

```
dym(x)
```

```
## [1] "2001-02-09"
```

hour, minute, and second functions

The **lubridate** package has functions to extract the hour, minute, and second values from a time

- Times are in the format **HH:MM:SS** and are stored as number of seconds since **00:00:00**
- Different time zones are also available

```
now()
```

```
## [1] "2025-07-15 21:30:22 EDT"
```

```
now("GMT")
```

```
## [1] "2025-07-16 01:30:22 GMT"
```

```
now() %>% hour()
```

```
## [1] 21
```

```
now() %>% minute()
```

```
## [1] 30
```

```
now() %>% second()
```

```
## [1] 22.49605
```


Time parser functions

The **hms** function converts a string into a time data type

There are also functions that can parse values that contain **both a date and a time**

```
x <- c("12:34:56")  
hms(x)
```

```
## [1] "12H 34M 56S"
```

```
x <- "Nov/2/2012 12:34:56"  
mdy_hms(x)
```

```
## [1] "2012-11-02 12:34:56 UTC"
```


Summary

- Dates and times are types of data that are often encountered in a data science project
- The **lubridate** package contains functions that can intuitively and efficiently parse dates and times